Library Management System – Execution Roadmap

Project Overview

Project Name: Knowledge HUB – Library Management System

Technology Stack:

- Backend: Java, Spring Boot, Spring Data JPA, Spring Security (JWT)
- Frontend: Angular
- Database: MySQL
- Caching: Redis
- Storage: AWS S3
- Tools: Postman, Git, Maven

Objective:
To build a full-stack, production-ready Library Management System with authentication, book inventory tracking, issue/return workflows, notifications, and scalable architecture.

Development Approach:
Backend-first, modular monolith initially, with gradual evolution into microservices.

Step 2: Execution Roadmap

This roadmap is divided into small, trackable daily activities. Each task includes:

- What to Code
- What to Learn
- What to Test

Phase 1: Project Setup and Foundation (Days 1–5)

Day 1: Project Initialization

What to Code:

- Create Spring Boot project using Spring Initializr
- Add dependencies: Web, JPA, MySQL Driver, Lombok
- Setup base package structure

What to Learn:

- Spring Boot project structure
- Dependency management using Maven

What to Test:

- Run application successfully
- Verify default endpoint

Day 2: Database Setup

What to Code:

- Install MySQL
- Create database: library_db
- Configure application.properties

What to Learn:

- JDBC connection properties
- Hibernate dialect and ddl-auto

What to Test:

- Application connects to DB

Day 3: Base Entity Setup

What to Code:

- Create BaseEntity (id, createdAt, updatedAt)
- Enable JPA auditing

What to Learn:

- @MappedSuperclass
- @EntityListeners

What to Test:

- Auto timestamp population

Day 4: User and Role Entity

What to Code:

- User entity
- Role entity
- Relationship mapping

What to Learn:

- OneToMany, ManyToOne

What to Test:

- Save and fetch user

Day 5: Repository Layer

What to Code:

- UserRepository
- RoleRepository

What to Learn:

- JpaRepository basics

What to Test:

- CRUD operations

Phase 2: Authentication Module (Days 6–12)

Day 6: Registration API

What to Code:

- DTOs for registration
- Service and Controller

What to Learn:

- DTO pattern

What to Test:

- Register user via Postman

Day 7: Password Encryption

What to Code:

- Configure BCrypt

What to Learn:

- Password hashing

What to Test:

- Password stored encrypted

Day 8: JWT Setup

What to Code:

- JWT utility class

What to Learn:

- Token generation and validation

What to Test:

- Generate token

Day 9: Login API

What to Code:

- Authentication logic

What to Learn:

- AuthenticationManager

What to Test:

- Login returns token

Day 10: Security Config

What to Code:

- Spring Security config

What to Learn:

- Filters and authentication flow

What to Test:

- Secure endpoints

Day 11: Role-Based Access

What to Code:

- Role restrictions

What to Learn:

- @PreAuthorize

What to Test:

- Admin vs User access

Day 12: OTP Email Integration

What to Code:

- SMTP config
- OTP logic

What to Learn:

- Java Mail Sender

What to Test:

- Email delivery

Phase 3: Book Module (Days 13–20)

Day 13: Book Entity

What to Code:

- Book entity

What to Learn:

- Entity design best practices

What to Test:

- Save book

Day 14: Genre Entity

What to Code:

- Genre table

What to Learn:

- Relationships

What to Test:

- Mapping validation

Day 15: Book Repository

What to Code:

- Repository methods

What to Learn:

- Custom queries

What to Test:

- Fetch books

Day 16: Add Book API

What to Code:

- Controller and service

What to Learn:

- Validation

What to Test:

- Add book via Postman

Day 17: Update/Delete APIs

What to Code:

- Update and delete

What to Learn:

- Transaction handling

What to Test:

- CRUD validation

Day 18: Search API

What to Code:

- Search by title/author

What to Learn:

- JPQL

What to Test:

- Search results

Day 19: Pagination

What to Code:

- Pageable APIs

What to Learn:

- Pagination concepts

What to Test:

- Page response

Day 20: Book Availability Logic

What to Code:

- Offline count logic

What to Learn:

- Business rules

What to Test:

- Availability toggle

Phase 4: Transaction Module (Days 21–28)

Day 21: Transaction Entity

What to Code:

- Issue/return entity

What to Learn:

- Date handling

What to Test:

- Persist data

Day 22: Issue Book API

What to Code:

- Issue logic

What to Learn:

- Service layer logic

What to Test:

- Reduce count

Day 23: Return Book API

What to Code:

- Return logic

What to Learn:

- State transitions

What to Test:

- Increase count

Day 24: Due Date Logic

What to Code:

- 12-day calculation

What to Learn:

- LocalDate

What to Test:

- Correct due date

Day 25: Fine Calculation

What to Code:

- Fine logic

What to Learn:

- Business calculations

What to Test:

- Fine computed

Day 26: Borrow History API

What to Code:

- History endpoint

What to Learn:

- DTO mapping

What to Test:

- Correct data

Day 27: Active Loans API

What to Code:

- Active books

What to Learn:

- Query optimization

What to Test:

- Accurate data

Day 28: Validation and Edge Cases

What to Code:

- Prevent invalid issue

What to Learn:

- Exception handling

What to Test:

- Edge scenarios

Phase 5: Notification Module (Days 29–32)

Day 29: Email Service

What to Code:

- Email utility

What to Learn:

- SMTP config

What to Test:

- Send email

Day 30: Issue Email Trigger

What to Code:

- Send on issue

What to Learn:

- Event handling

What to Test:

- Email sent

Day 31: Reminder Scheduler

What to Code:

- Cron job

What to Learn:

- Scheduling in Spring

What to Test:

- Trigger logic

Day 32: Overdue Notification

What to Code:

- Overdue alerts

What to Learn:

- Scheduled queries

What to Test:

- Alerts working

Phase 6: Request Module (Days 33–35)

Day 33: Request Entity

Day 34: Request API

Day 35: Admin View Requests

Phase 7: Redis Caching (Days 36–38)

Day 36: Redis Setup

Day 37: Cache Book List

Day 38: Cache Search

Phase 8: AWS S3 Integration (Days 39–41)

Day 39: S3 Setup

Day 40: Upload API

Day 41: Download API

Phase 9: Frontend Integration (Days 42–55)

- Authentication UI
- Book UI
- Dashboard

Phase 10: Testing and Deployment (Days 56–65)

- API Testing
- Bug Fixing
- Deployment

Final Outcome

A complete, production-ready Library Management System with strong backend architecture, real-world features, and scalable design.

This roadmap ensures consistent progress, deep understanding, and complete ownership of the system.